An AWS Fog Pipeline for Data Center Environmental Monitoring

Nithin
Student ID: X25125338
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25125338@student.ncirl.ie

***Abstract*—A fog and edge computing pipeline for data center environmental monitoring is presented in this paper, covering its design, implementation, and deployment to a real-world cloud environment. The system simulates five distinct sensor types (temperature, humidity, airflow, power load, and dust density) across two server halls, each independently configurable for sampling and dispatch frequency. A Node.js fog node buffers incoming readings in a fixed-size ring buffer, performs windowed aggregation and threshold-based alerting, and dispatches each window's results as a batched message. The backend is entirely serverless and split across two independently deployed AWS Lambda functions. The first is triggered by an Amazon SQS queue and writes aggregated windows to Amazon DynamoDB, while the second performs internal request routing and sits behind a real Amazon API Gateway REST API. This design avoids a dashboard server that calls AWS services directly. The dashboard's static assets are served from Amazon S3 and call the API Gateway endpoint, with no intermediary server. Rather than validating the system only against a local emulator, the full pipeline was deployed to and tested on a real AWS account, a step that exposed defects local testing could not reveal: among them, bugs in credential handling that appear only under temporary AWS credentials, and a DynamoDB pagination issue that quietly undercounted results on large table scans. Each layer's architectural decisions are critically analysed here, alongside the defects uncovered through real deployment and the broader lessons they offer for building cloud-native fog computing systems.**

***Keywords*—fog computing, edge computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, data center monitoring**

I. Introduction

Data centers depend on continuous environmental monitoring to protect the hardware they house. An undetected overheating server, a humidity swing that risks condensation on sensitive equipment, restricted airflow around a rack, an unexpected power-load spike, or rising dust density between scheduled inspections can each trigger a hardware failure or a costly unplanned outage. This is precisely the class of condition wireless sensor deployments in real data centers have been built to catch before it escalates [1], [2]. Where continuous sensing is absent and a facility instead relies on fixed inspection rounds, these conditions surface only after they have already occurred, the very moment a condition-driven system should have flagged them instantly. Internet of Things sensing, combined with fog and edge processing close to the sensors themselves, allows a facilities team to evaluate every reading against explicit threshold rules as it arrives; there is no need to wait for the next scheduled walk-through.

Five essential characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service) form the basis of the National Institute of Standards and Technology's definition of cloud computing [3]; fog computing extends that model instead of replacing it. In the formulation offered by Bonomi et al. [4], fog computing extends cloud services out to the network edge, a platform distinguished by low latency, wide geographic distribution, and the ability to support very large numbers of nodes. That definition shapes this project directly: a fog node situated alongside the sensors carries out windowed aggregation and threshold evaluation locally, so it is the resulting aggregate-plus-alerts payload, not the raw reading stream, that crosses into the cloud.

What follows documents how a fog and edge computing pipeline for this domain was designed and built. Three goals guided the system's design. The first was a sensor and fog layer capable of generating data across five distinct sensor types, each with sampling and dispatch rates configurable independently, and a fog node that buffers, aggregates, and evaluates that data before passing it on. The second was a scalable backend, built on managed queue and function-as-a-service mechanisms, able to process the fog node's output and drive a responsive dashboard across all sensor types. The third was actually deploying and testing that backend on a public cloud platform, rather than settling for a local simulation.

The rest of the paper proceeds as follows. Section II lays out the system's architecture layer by layer, covering the separately deployed API backend that sets this project's design apart, the alternative architectures weighed against it, and the security posture that results. Implementation detail follows in Section III: the software stack, the automated test suite, the continuous integration pipeline, and what the real AWS deployment revealed once run. Section IV then turns to evaluation, grounding the assessment in measured evidence rather than stated intent. The paper closes in Section V with a critical reflection on the project and the directions future work might take.

TABLE I
ENVIRONMENTAL ALERT RULES

Sensor Type	Rule	Alert Key
temperature_c	avg $>$ 27°C	overheat_risk
humidity_pct	avg $>$ 60%	condensation_risk
humidity_pct	avg $<$ 20%	static_discharge_risk
airflow_cfm	avg $<$ 400 CFM	insufficient_cooling
power_load_kw	avg $>$ 130 kW	capacity_warning
dust_density_ugm3	avg $>$ 50 ug/m3	air_quality_risk

II. SYSTEM ARCHITECTURE AND DESIGN

A. Sensor and Fog Layer

The simulated deployment models two server halls, hall-1 and hall-2, each instrumented with five sensor types: temperature_c, humidity_pct, airflow_cfm, power_load_kw, and dust_density_ugm3. Each of the ten resulting sensor processes takes two environment variables, SAMPLE_INTERVAL and DISPATCH_INTERVAL, configurable independently of one another: generation frequency and dispatch rate are therefore two separate knobs, not a single value aliased to both.

The fog node buffers incoming readings in a fixed-size ring buffer keyed by (sensor_type, site_id). Each key holds a plain array of RING_CAPACITY (256) slots with a manually tracked write index that advances modulo the capacity, a bounded circular-buffer structure for streaming data in place of an unbounded growing list [5], [6]. On every window tick, the fog node reconstructs each ring's contents in oldest-first order and computes the window's minimum, maximum, average, latest value, and reading count. It then evaluates the aggregate against the domain-specific threshold rules listed in Table I and resets the ring for the next window. The resulting aggregate-plus-alerts payload is dispatched to an Amazon SQS queue through an EventEmitter-decoupled publisher. Window-close and the SQS send remain two separate concerns joined only by an emitted event, echoing the general publish-subscribe pattern of separating event producers from consumers in space, time, and synchronisation [7], [8]. The publisher always issues a batched SendMessageBatchCommand, chunked at SQS's ten-entry ceiling, so every group closed in one flush cycle goes out together whenever more than one is ready.

B. Scalable Backend Layer

Managed, scalable AWS primitives make up the entire backend layer. A standard Amazon SQS queue, dce-hall-agg, sits between the fog node's dispatch rate and the backend's own processing rate, decoupling the two. From there, an event-source mapping automatically invokes the dce-processor Lambda function whenever a message arrives on that queue, and each aggregated window it receives gets written into the dce-readings DynamoDB table. sensor_type serves as the table's partition key, paired with a range key built as a composite sort key, window_end#site_id. That composite mirrors a philosophy Amazon's own eventually-consistent, highly available key-value store established: query patterns built around a narrow set of known key lookups, not ad-hoc relational joins [9]. The composite shape is not incidental. Had the sort key been window_end on its own, an aggregate for the same sensor type closing in the same flush cycle for both hall-1 and hall-2 would land on one identical primary key, and whichever of the two writes arrived second would quietly clobber the first. Suffixing the sort key with the site identifier keeps the two halls' concurrent writes apart, with no extra index required to do it.

Cold-start latency (the initialisation delay that occurs when an invocation arrives with no warm execution environment ready for it) is a frequently cited drawback of Lambda-based architectures, and one that empirical studies of commercial FaaS platforms have measured directly [10]. This is a cost of the FaaS approach, but it is acceptable here for the same reason it is acceptable for the ingestion path: the fog-to-processor path is asynchronous and queue-triggered, not a user-facing request/response path. An occasional cold-start delay when a reading is durably stored is therefore not a user-visible page load.

C. A Separately Deployed API Backend Behind a Real API Gateway

Unlike a design in which the dashboard's backend process calls DynamoDB, SQS, and Lambda directly, this project deploys a second, entirely separate Lambda function, dce-api. The function performs internal path-and-method routing against an ordered [method, regex, handler] table, which can be unit-tested with fake AWS clients and no Lambda runtime at all. dce-api is fronted by a real Amazon API Gateway REST API: a {proxy+} resource under the root, with an ANY-method AWS_PROXY integration. Every request the dashboard makes to any /api/* path is therefore proxied verbatim to dce-api, which then decides internally how to answer it. This centralises cross-cutting request handling behind a single gateway rather than exposing each backend capability as a separately reachable endpoint [11]. dce-api answers five endpoints: GET /api/readings, GET /api/halls (and /api/halls/:hallId for an individual hall), GET /api/health, GET /api/backend-stats, and GET /api/thresholds. The last of these is proxied from the fog node's /thresholds endpoint, letting a client retrieve the live alert rules without duplicating them.

D. Dashboard Layer and Its Real-AWS Adaptation

The dashboard was designed from the outset as a deliberately minimal reverse proxy, not a REST API. Its Node.js HTTP server resolves the API Gateway's invoke URL exactly once at startup and forwards every /api/* request to it verbatim, holding no AWS SDK imports and no route table. Locally, this same server also serves the dashboard's static assets on one port. For the real AWS deployment, this reverse-proxy role was retired rather than ported to a second EC2 process. An Amazon S3 bucket now serves the dashboard's HTML, CSS, and JavaScript over its own HTTPS endpoint, with the frontend calling API Gateway's invoke URL directly rather than through any intermediary. This URL is configured through a meta tag in place of a hard-coded value, and the

dce-api Lambda's JSON responses carry an Access-Control-Allow-Origin header for the resulting cross-origin request.

This reflects a known constraint: plain HTTP served from a raw EC2 public IP address on a non-standard port is precisely the traffic shape that campus and corporate WiFi networks tend to block as a matter of policy. Keeping the dashboard on EC2 at all would have brought that same reachability risk back in, which the fully serverless path sidesteps completely.

E. Deployment Topology

Fig. 1 lays out the topology that results. Docker containers running on a single EC2 instance host the sensor processes and the fog node, with AWS Systems Manager Session Manager standing in for SSH as the sole means of administration; consequently, no key pair exists and the instance's security group opens no inbound SSH port. From there, aggregated windows leave the fog node for an SQS queue, which triggers dce-processor, which in turn writes to DynamoDB. Behind API Gateway, dce-api answers the dashboard's REST calls by reading from that DynamoDB table plus three further sources: the queue's depth, dce-processor's own metadata, and the fog node's health/thresholds endpoints. The dashboard's static frontend follows a different path entirely, served independently from Amazon S3 and reached by the browser directly. What this topology achieves is putting every backend component behind a managed, serverless boundary, every one, that is, except the sensor and fog tier itself.

F. Critical Analysis of Alternative Architectures

A number of alternative designs came under consideration and were ultimately set aside, each for a reason that can be stated plainly. A single Lambda function handling both queue-triggered ingestion and API-Gateway-fronted request serving was considered in place of two separate functions, and rejected. Ingestion and query serving have different invocation triggers and different concurrency and timeout characteristics. Coupling them in one function would mean a slow or failing query path could starve the ingestion path of concurrency, or vice versa. This is exactly the failure mode that splitting the two functions is meant to avoid.

An unbounded, per-key growing array was considered in place of the fixed-size ring buffer for fog-side buffering, and rejected in favour of a bounded structure: an unbounded buffer risks unconstrained memory growth if a downstream flush is ever delayed. A fixed-capacity ring buffer with a wrapping write index guarantees a hard upper bound on memory use regardless of how long a window runs, at the cost of silently discarding the oldest readings once that bound is exceeded. This is an accepted trade for a workload whose window length is short relative to its sampling rate.

A synchronous dashboard backend that calls AWS services in-process was considered in place of the decoupled, API-Gateway-fronted dce-api Lambda. That approach was rejected because an always-running process that imports the AWS SDK directly reintroduces the same continuous compute cost and single-process failure mode the rest of the backend deliberately avoids: dce-api behind API Gateway scales per request and costs nothing when idle, whereas a synchronous in-process server has to stay running regardless of traffic, pushing scaling and failure isolation for the dashboard tier onto the developer instead of the platform.

One further option, converting the fog node and sensors to a scheduled-Lambda model so that the whole system, not just the backend, became serverless, was weighed and set aside for later rather than rejected outright. Lambda's stateless, time-limited invocation model does not accommodate a continuous sensor loop or a stateful ring buffer without the buffering logic being redesigned first, and in any case this project's cloud-deployment scope was always the backend layer, not the sensor and fog layer. The redesign such a conversion would demand outweighed what the remaining project time could support, so it appears in Section V as future work instead.

G. Security Considerations

No key pair was ever provisioned for the EC2 instance, and its security group has no inbound rule for port 22: administration runs solely through AWS Systems Manager Session Manager, which leaves no SSH attack surface whatsoever. The single inbound rule that is open, on port 8000, exists only so dce-api can reach the fog node's health and threshold endpoints, and what it returns is read-only, non-sensitive status information, nothing that amounts to data or control-plane access.

The account's service-control policies block a student from minting new, more narrowly scoped IAM roles, so both Lambda functions run instead under the AWS Academy Learner Lab's shared LabRole, a genuine constraint of the environment, not a choice made on design grounds. A production deployment would give dce-processor and dce-api their own least-privilege roles instead of sharing one: dce-processor would need nothing beyond sqs:ReceiveMessage/DeleteMessage and dynamodb:PutItem, while dce-api would need only read-oriented access to DynamoDB, SQS, and Lambda metadata. That gap is stated plainly here rather than left implicit. Separately, the dashboard's REST API carries no authentication by design, since all it exposes is aggregated, non-personal environmental data for a facility, not the kind of data where access control would meaningfully matter at this project's scale.

III. Implementation

A. Software Components and Libraries

Node.js 20 is what the sensor, fog, processor, and dce-api components are all built on, written in plain CommonJS with no TypeScript build step, and every interaction with AWS goes through the official AWS SDK for JavaScript v3. None of them reach for a web framework like Express, either: Node's built-in http module does that job instead, keeping the dependency surface small on purpose. The dashboard's trend visualisation is rendered client-side with Chart.js. Rather than pulling it from a content-delivery network, the library ships vendored locally, leaving the page with no third-party

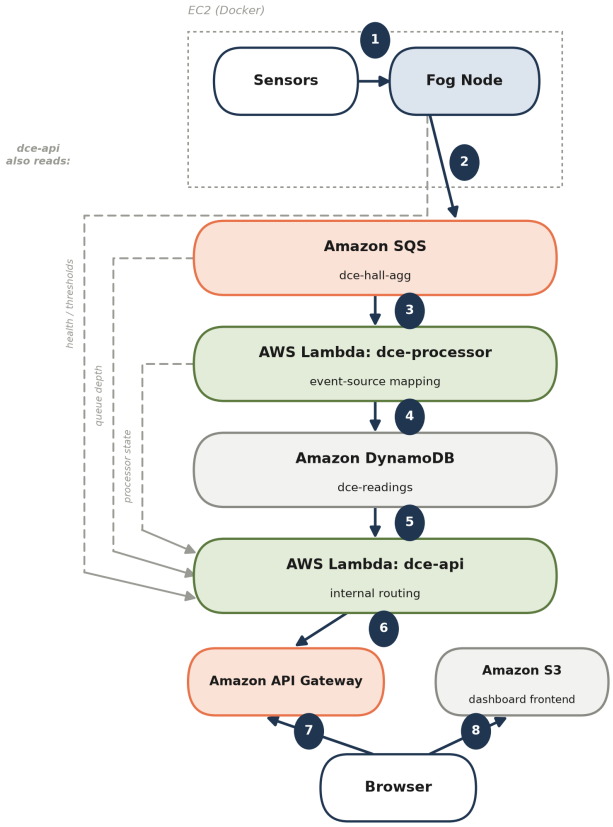

Fig. 1. System topology at deployment: EC2 hosts the sensors and fog node, the backend (SQS, the two Lambda functions, DynamoDB, and API Gateway) is entirely serverless, and S3 serves the frontend.

runtime dependency. For local development and continuous integration alike, Docker Compose runs alongside LocalStack, an open-source AWS cloud emulator, letting the same SQS, DynamoDB, and Lambda calls used in production run day to day without ever touching a real AWS account.

B. Testing Strategy

Node's built-in test runner drives the automated test suite for every module, and rather than reaching for a mocking library, that suite exercises hand-written fake AWS client objects, which means each unit test can assert on the precise shape of the request a given AWS SDK call actually sends. The ring buffer's test suite specifically covers wraparound behaviour, including a case where writes exceed capacity mid-window and only the last RING_CAPACITY readings survive. It also verifies that every ring's slots, write index, and count are reset to empty on every window tick. The EventEmitter-decoupled publisher is tested for exactly one SendMessageBatchCommand call carrying multiple Entries when more than one group closes in the same flush cycle. dce-api's internal router is tested by dispatching method-and-path-regex matches against fake AWS clients directly, with no Lambda runtime or API Gateway involved at all.

A load-testing script goes further than unit tests, firing a configurable number of synthetic messages at the real queue in a burst and then checking two things. Immediately after the burst, it confirms queue depth is non-zero, evidence the messages actually reached SQS rather than vanishing silently. Then, allowing a timeout window, it checks that depth has since dropped, whether fully drained or just measurably lower, a tolerance built in specifically so that Lambda's concurrency ceiling doesn't get mistaken for a hard failure when the drain simply takes longer than expected.

C. Continuous Integration and Deployment

Every push to the repository triggers a GitHub Actions workflow that runs the complete automated test suite across all five modules. A follow-on job then spins up the whole stack with Docker Compose against LocalStack, running a dedicated end-to-end script that submits synthetic readings through the live pipeline and checks they arrive in DynamoDB with the sort_key shape expected. Whatever the outcome, the stack comes back down afterward, so no failed run is left holding a stale container open for whatever runs next.

Because the Learner Lab platform stops the EC2 instance whenever a session ends, a second GitHub Actions workflow, triggered independently of the test workflow, exists purely

to bring the EC2-hosted fog and sensor tier back up at the start of a fresh session. Before it touches anything, it pulls AWS credentials from encrypted GitHub repository secrets and confirms they resolve to the expected account. Only after that check passes does it start the instance, wait for the Systems Manager agent on it to register, and confirm the fog node is reachable both directly and through dce-api's gateway field; success is reported only once both checks come back clean.

D. Real AWS Deployment and Defects Discovered

Every piece of the system (the sensor and fog tier, the SQS queue, both Lambda functions, the DynamoDB table, the API Gateway endpoint, and the S3-hosted frontend) ended up deployed to a real AWS account, an AWS Academy Learner Lab account in the us-east-1 region. That meant testing the backend against an actual public cloud platform, not a local emulator alone, and doing so brought two classes of defect to light that the LocalStack-backed local test suite had missed entirely.

The first defect showed up in three separate files spread across the fog, processor, and dce-api components, each of which built its AWS SDK client credentials from a condition that happened to hold true whether the code was running locally or on real AWS: if an AWS access key environment variable existed at all, a hardcoded, LocalStack-only credential pair got applied; one of the three files did this with no conditional check whatsoever. The trouble is that real AWS Lambda and EC2 environments populate that exact same environment variable automatically, filling it with genuine temporary, session-token-bearing credentials, so this check quietly threw those real credentials away in favour of the hardcoded pair on every actual deployment, meaning every DynamoDB and SQS call would have failed authentication in production even though the LocalStack-backed tests all passed cleanly. Fixing it meant gating the hardcoded override behind something that is genuinely unique to the local-emulator case: an explicit LocalStack endpoint override. Each of the three files expressed this gating logic with a different code shape, not the same patch copy-pasted three times.

Second, a COUNT-select Scan call is how dce-api's backend-stats endpoint reported the total item count stored in DynamoDB, a call that only counts the single page it actually reads, capped at roughly 1MB of table data. Any table larger than that returns a LastEvaluatedKey signalling more to fetch, and unless a further scan follows it, the reported count quietly falls short of the real total. At this project's data volume the bug never manifested visibly, but it was caught and fixed regardless, since a COUNT-select Scan's single-page limit is a well-known DynamoDB pagination pitfall regardless of workload. The fix follows LastEvaluatedKey across pages and sums every page's Count, with two new tests covering the multi-page-summing and scan-failure cases.

Verification of both fixes happened against the live deployment itself, not just in theory: redeploying and then successfully completing a DynamoDB write and SQS receive end to end confirmed the credential fix, while the pagination fix was confirmed by watching the deployed table return a correct items_in_table count directly, rather than taking the unit test suite's word for it. Everything described here (source code and fixes alike) sits at https://github.com/nithinveshala/fog-and-edge, alongside a readme.txt that walks through installation, configuration, and this section's deployment history.

TABLE II
AUTOMATED TEST SUITE BY MODULE

Module	Tests	Focus
sensors	12	dual-rate sampling/dispatch independence
fog	46	ring buffer wraparound, windowed aggregation, alerting
backend/processor	9	DynamoDB item shape, sort-key logic
backend/api	37	internal routing, DynamoDB pagination, thresholds proxy
backend/dashboard	10	reverse-proxy behaviour, invoke-URL resolution

IV. EVALUATION

A. Automated Test Coverage

As of the final verified deployment, the automated test suite breaks down as shown in Table II: all 114 tests pass, both against the LocalStack-backed development environment and, where the suite exercises real client-construction logic, against the fixes confirmed on the real AWS deployment covered in Section III-D.

B. Live Load Test

In a burst load test, 300 synthetic messages went straight to the real, deployed dce-hall-agg queue in just 5.47 seconds, a rate of roughly 55 messages per second. The test used synthetic sensor-type names distinct from the five real sensor types, so the burst could never land in the DynamoDB partitions the live dashboard reads from. Amazon SQS's ApproximateNumberOfMessages and ApproximateNumberOfMessagesNotVisible attributes reported a queue depth of zero immediately after the burst completed, a result that would ordinarily read as a failed send. A direct DynamoDB query against the five synthetic sensor-type partitions, however, confirmed that all 300 messages had been received, processed, and written by dce-processor before the follow-up check even ran.

Rather than pointing to a broken pipeline, this lines up with a documented property of real SQS: the queue's approximate-count attributes are explicitly eventually consistent, and under-reporting immediately after a burst is exactly the behaviour that consistency model predicts, a behaviour LocalStack's more immediately consistent emulation never reproduces. Nothing short of testing against the real service would have surfaced it in this evaluation.

C. Live Deployment Health

Two polls of the deployed API's /api/health endpoint, fifteen seconds apart, returned freshest_age_seconds values of 7.429

Fig. 2. The deployed dashboard, served from Amazon S3, showing live per-hall readings, window-average trends for all five sensor types, and the four pipeline health indicators.

and then 3.357, with items_in_table counts that behaved like a live, continuously updating pipeline rather than anything static or cached. Rather than take the application's own self-reported status at face value, every AWS resource involved (the DynamoDB table, the SQS queue, both Lambda functions, the API Gateway API, and the EC2 instance) was checked independently and confirmed healthy through direct AWS CLI calls. Fig. 2 shows the S3-served dashboard rendering the same live pipeline in the browser during this verification.

At this project's data volume, every backend component apart from the EC2 instance stays within AWS's request-based free-tier allowances operationally. The EC2 instance is the sole resource billed continuously, but it can be switched off between demo sessions without touching either the dashboard or its API: neither depends on the instance running, save for the gateway field inside /api/health and the arrival of freshly generated sensor data.

D. Limitations

Rather than leave them unstated, three limitations of this evaluation deserve to be named directly. The fog and sensor tier is the first: it still runs on one single EC2 instance, which makes it a single point of failure specifically for that tier, even while everything behind it is fully redundant, AWS-managed infrastructure. Section II-F's critical analysis explains why moving this tier to a fully serverless model was deferred rather than tried, but that does not make the resulting availability gap any less real or unresolved. Session-based rather than persistent is the second limitation: the AWS Academy Learner Lab account this deployment used could not support the sustained, multi-day uptime evaluation a production deployment would call for, so what this section evaluates is verified behaviour within a single continuous session, not long-run reliability. Third, one broadly scoped IAM role is shared between both Lambda functions rather than each holding a least-privilege role of its own, a constraint imposed by the Learner Lab account, discussed further in Section II-G, and not a deliberate security choice. This reflects a genuine limit on what could be independently verified about the deployment's security posture; it is not evidence that a least-privilege alternative was tried and found wanting.

V. Conclusion

Building a complete fog and edge computing pipeline for data center environmental monitoring was this project's goal from the outset: a pipeline made up of a configurable sensor and fog layer, a scalable cloud backend divided across two independently deployed Lambda functions, and testing and deployment on a genuine public cloud platform. Alongside implementation, the project set out to critically weigh each layer's architectural decisions against the alternatives on offer, and all three layers were, in the end, implemented, tested, and deployed to a real AWS account rather than left to a local simulation.

If this project has one significant finding, it is methodological rather than architectural: however thorough testing against a cloud emulator might be, it cannot substitute for testing against the real target platform. The credential-handling bug and the DynamoDB pagination bug, both uncovered only once real AWS deployment happened, sailed cleanly through the full local test suite and a full LocalStack-backed integration test alike, and would have gone out the door undetected had the project stopped short at local simulation. Seen from a different angle, the live load test's eventual-consistency finding makes the same case: a local emulator that doesn't reproduce a platform behaviour, such as SQS's approximate-count consistency model, means a passing local test simply cannot exercise that behaviour at all.

Writing application code turned out not to be the hard part of this project: discovering the AWS Academy Learner Lab account's platform and IAM constraints was. Design decisions were shaped outright by several of these constraints, the shared LabRole and a session-based rather than persistent account lifetime chief among them; neither was merely incidental. Among the directions future work could take is converting the fog and sensor tier itself away from a continuously running EC2 instance, toward a fully event-driven, scheduled-Lambda model instead. If the account restrictions were lifted, future work would also provision two separate least-privilege IAM roles for dce-processor and dce-api instead of the single shared LabRole this deployment was constrained to.

References

[1] C.-J. M. Liang, J. Liu, L. Luo, A. Terzis, and F. Zhao, "RACNet: A high-fidelity data center sensing network," in *Proc. 7th ACM Conf. Embedded Networked Sensor Systems (SenSys '09)*, Berkeley, CA, USA, 2009, pp. 15–28.

[2] M. G. Rodriguez, L. Ortiz Uriarte, Y. Jia, K. Yoshii, R. Ross, and P. H. Beckman, "Wireless sensor network for data-center environmental monitoring," in *Proc. 2011 Fifth Int. Conf. Sensing Technology (ICST)*, Palmerston North, New Zealand, 2011, pp. 533–537.

[3] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, 2011.

[4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st ACM Mobile Cloud Computing Workshop (MCC '12)*, Helsinki, Finland, 2012, pp. 13–16.

[5] J. Giacomoni, T. Moseley, and M. Vachharajani, "FastForward for efficient pipeline parallelism: A cache-optimized concurrent lock-free queue," in *Proc. 13th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP '08)*, 2008, pp. 43–52.

[6] L. Lamport, "Concurrent reading and writing," *Commun. ACM*, vol. 20, no. 11, pp. 806–811, 1977.

[7] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Comput. Surv.*, vol. 35, no. 2, pp. 114–131, Jun. 2003.

[8] A. Carzaniga, D. S. Rosenblum, and A. L. Wolf, "Design and evaluation of a wide-area event notification service," *ACM Trans. Comput. Syst.*, vol. 19, no. 3, pp. 332–383, Aug. 2001.

[9] G. DeCandia *et al.*, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, 2007, pp. 205–220.

[10] M. Shahrad, J. Balkind, and D. Wentzlaff, "Architectural implications of function-as-a-service computing," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO-52)*, Columbus, OH, USA, 2019, pp. 1063–1075.

[11] A. Akbulut and H. G. Perros, "Software versioning with microservices through the API gateway design pattern," in *Proc. 2019 9th Int. Conf. Advanced Computer Information Technologies (ACIT)*, Ceske Budejovice, Czech Republic, 2019, pp. 289–292.